RAJIV GANDHI INSTITUTE OF PETROLEUM TECHNOLOGY, JAIS, AMETHI

Department of Computer Science and Engineering

Compiler Design (CS312)

By

Dr. Kalka Dubey

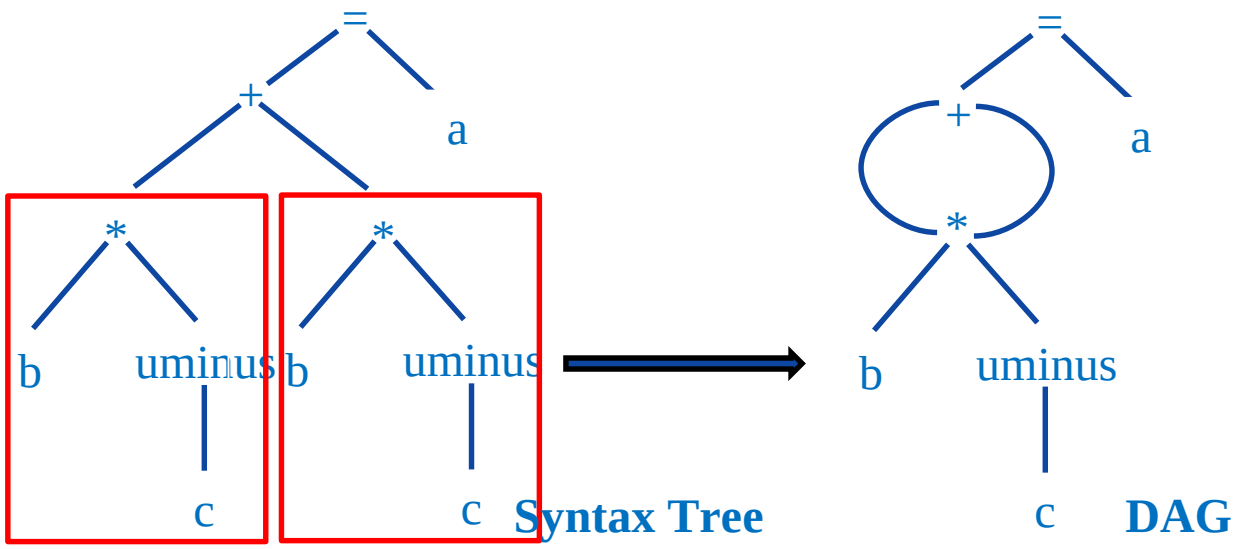
Different intermediate forms

- **Different forms of intermediate code are:**

1. Abstract syntax tree
2. Postfix notation
3. Three address code

Abstract syntax tree & DAG

- A syntax tree depicts the natural hierarchical structure of a source program.
- A DAG (Directed Acyclic Graph) gives the same information but in a more compact way because common sub-expressions are identified.
- Ex: a = b * -c+b*-c



Postfix Notation

- Postfix notation is a linearization of a syntax tree.
- In postfix notation the operands occurs first and then operators are arranged.
- Ex: **(A + B) * (C + D)**

- Ex: **(A + B) * C**

Postfix notation: A B + C D + *

- Ex: **(A * B) + (C * D)**

Postfix notation: A B + C *

Postfix notation: A B * C D * +

Three address code

- Three address code is a sequence of statements of the general form,

  a:= b op c

- Where a, b or c are the operands that can be names or constants and op stands for any operator.
- Example: a = b + c + d

  t_1=b+c

  t_2=t_1+d

  a= t_2

- Here t_1 and t_2 are the temporary names generated by the compiler.
- There are at most three addresses allowed (two for operands and one for result). Hence, this representation is called three-address code.

Different Representation of Three Address Code

- There are three types of representation used for three address code:
  1. Quadruples
  2. Triples
  3. Indirect triples
- Ex: x= -a*b + -a*b

t_1= - a
t_2 = t_1 * b
t_3= - a
t_4 = t_3 * b
t_5 = t_2 + t_4
x= t_5

Three Address Code

Quadruple

- The quadruple is a structure with at the most four fields such as op, arg1, arg2 and result.
- The op field is used to represent the internal code for operator.
- The arg1 and arg2 represent the two operands.
- And result field is used to store the result of an expression.

---

x= -a*b + -a*b

t_1= - a

t_2 = t_1 * b

t_3= - a

t_4 = t_3 * b

t_5 = t_2 + t_4

x= t_5

Quadruple

No.	Operator	Arg1	Arg2	Result
(0)				
(1)				
(2)				
(3)				
(4)				
(5)				

Triple

- To avoid entering temporary names into the symbol table, we might refer a temporary value by the position of the statement that computes it.
- If we do so, three address statements can be represented by records with only three fields: op, arg1 and arg2.

Quadruple

No.	Operator	Arg1	Arg2	Result
(0)	uminus	a		t_1
(1)	*	t_1	b	t_2
(2)	uminus	a		t_3
(3)	*	t_3	b	t_4
(4)	+	t_2	t_4	t_5
(5)	=	t_5		x

Triple

No.	Operator	Arg1	Arg2
(0)			
(1)			
(2)			
(3)			
(4)			
(5)			

Indirect Triple

- In the indirect triple representation the listing of triples has been done. And listing pointers are used instead of using statement.

- This implementation is called indirect triples.

Triple

No.	Operator	Arg1	Arg2
(0)	uminus	a	
(1)	*	(0)	b
(2)	uminus	a	
(3)	*	(2)	b
(4)	+	(1)	(3)
(5)	=	x	(4)

Indirect Triple

	Statement
(0)	(14)
(1)	(15)
(2)	(16)
(3)	(17)
(4)	(18)
(5)	(19)

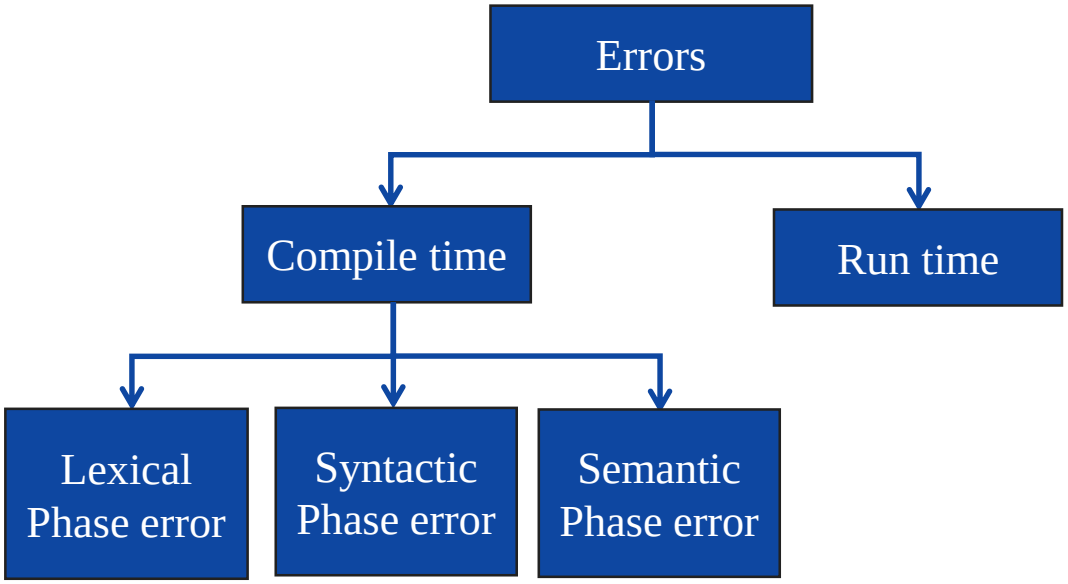
No.	Operator	Arg1	Arg2
(0)	uminus	a	
(1)	*		b
(2)	uminus	a	
(3)	*		b
(4)	+		
(5)	=	x	

Exercise

Write quadruple, triple and indirect triple for following:

1. -(a*b)+(c+d)
2. a*-(b+c)
3. x=(a+b*c)^(d*e)+f*g^h
4. g+a*(b-c)+(x-y)*d

Types of Errors



Lexical error

- Lexical errors can be detected during lexical analysis phase.
- Typical lexical phase errors are:
  1. Spelling errors
  2. Exceeding length of identifier or numeric constants
  3. Appearance of illegal characters
- Example:

```
fi ( )
{
}
```

- In above code 'fi' cannot be recognized as a misspelling of keyword ***if*** rather lexical analyzer will understand that it is an identifier and will return it as valid identifier.
- Thus misspelling causes errors in token formation.

Syntax error

- Syntax error appear during syntax analysis phase of compiler.
- Typical syntax phase errors are:
  1. Errors in structure
  2. Missing operators
  3. Unbalanced parenthesis
- The parser demands for tokens from lexical analyzer and if the tokens do not satisfies the grammatical rules of programming language then the syntactical errors get raised.
- Example:

printf(“Hello World !!!”) ← **Error: Semicolon missing**

Semantic error

- Semantic error detected during semantic analysis phase.
- Typical semantic phase errors are:
  1. Incompatible types of operands
  2. Undeclared variable
  3. Not matching of actual argument with formal argument
- Example:

  id1=id2+id3*60 (Note: id1, id2, id3 are real)

(Directly we can not perform multiplication due to incompatible types of variables)

Error recovery strategies

- There are mainly four error recovery strategies:
  1. Panic mode
  2. Phrase level recovery
  3. Error production
  4. Global generation

Panic mode

- In this method on discovering error, the parser discards input symbol one at a time. This process is continued until one of a designated set of synchronizing tokens is found.
- Synchronizing tokens are delimiters such as semicolon or end.
- These tokens indicate an end of the statement.
- If there is less number of errors in the same statement then this strategy is best choice.

```
fi ( )
{
}
```

← Scan entire line otherwise scanner will return fi as valid identifier

Phrase level recovery

- In this method, on discovering an error parser performs local correction on remaining input.

- The local correction can be:
  1. Replacing comma by semicolon
  2. Deletion of semicolons
  3. Inserting missing semicolon

- This type of local correction is decided by compiler designer.

- This method is used in many error-repairing compilers.

Error production

- If we have good knowledge of common errors that might be encountered, then we can augment the grammar for the corresponding language with error productions that generate the erroneous constructs.

- Then we use the grammar augmented by these error production to construct a parser.

- If error production is used then, during parsing we can generate appropriate error message and parsing can be continued.

Global correction

- Given an incorrect input string x and grammar G, the algorithm will find a parse tree for a related string y, such that number of insertions, deletions and changes of token require to transform x into y is as small as possible.

- Such methods increase time and space requirements at parsing time.

- Global correction is thus simply a theoretical concept.

Issues in design of Code Generator

- **Issues in Code Generation are:**
  1. Input to code generator
  2. Target program
  3. Memory management
  4. Instruction selection
  5. Register allocation
  6. Choice of evaluation
  7. Approaches to code generation

Input to code generator

- Input to the code generator consists of the intermediate representation of the source program.
- Types of intermediate language are:
  1. Postfix notation
  2. Three address code
  3. Syntax trees or DAGs
- The detection of semantic error should be done before submitting the input to the code generator.
- The code generation phase requires complete error free intermediate code as an input.

Target program

- The output may be in form of:

1. **Absolute machine language:** Absolute machine language program can be placed in a memory location and immediately execute.

2. **Relocatable machine language:** The subroutine can be compiled separately. A set of relocatable object modules can be linked together and loaded for execution.

3. **Assembly language:** Producing an assembly language program as output makes the process of code generation easier, then assembler is require to convert code in binary form.

Memory management

- Mapping names in the source program to addresses of data objects in run time memory is done cooperatively by the front end and the code generator.
- We assume that a name in a three-address statement refers to a symbol table entry for the name.
- From the symbol table information, a relative address can be determined for the name in a data area.

Instruction selection

- Example: the sequence of statements

  a := b + c

  d := a + e

- would be translated into

  MOV b, R0

  ADD c, R0

  MOV R0, a

  MOV a, R0

  ADD e, R0

  MOV R0, d

- So, we can eliminate redundant statements.

MOV b, R0

ADD c, R0

ADD e, R0

MOV R0, d

Register allocation

- The use of registers is often subdivided into two sub problems:
- During register allocation, we select the set of variables that will reside in registers at a point in the program.
- During a subsequent register assignment phase, we pick the specific register that a variable will reside in.
- Finding an optimal assignment of registers to variables is difficult, even with single register value.
- Mathematically the problem is NP-complete.

Choice of evaluation

- The order in which computations are performed can affect the efficiency of the target code.
- Some computation orders require fewer registers to hold intermediate results than others.
- Picking a best order is another difficult, NP-complete problem.

Approaches to code generation

- The most important criterion for a code generator is that it produces correct code.

- The design of code generator should be in such a way so it can be implemented, tested, and maintained easily.

Target machine

- We will assume our target computer models a three-address machine with:
  1. load and store operations
  2. computation operations
  3. jump operations
  4. conditional jumps

- The underlying computer is a byte-addressable machine with n general-purpose registers, $R_0, R_1, \ldots, Rn$

Addressing Modes

Mode	Form	Address	Extra cost
Absolute	M	M	1
Register	R	R	0
Indexed	k(R)	k +contents(R)	1
Indirect register	*R	contents(R)	0
Indirect indexed	*k(R)	contents(k + contents(R))	1

Basic Blocks

- A basic block is a sequence of consecutive statements in which flow of control enters at the beginning and leaves at the end without halt or possibility of branching except at the end.

- The following sequence of three-address statements forms a basic block:

```
t1 := a*a
t2 := a*b
t3 := 2*t2
t4 := t1+t3
t5 := b*b
t6 := t4+t5
```

Algorithm: Partition into basic blocks

Input: A sequence of three-address statements.

Output: A list of basic blocks with each three-address statement in exactly one block.

Method:

1. We first determine the set of **leaders**, for that we use the following rules:
    I. The first statement is a leader.
    II. Any statement that is the target of a conditional or unconditional goto is a leader.
    III. Any statement that immediately follows a goto or conditional goto statement is a leader.
2. For each leader, its basic block consists of the leader and all statements up to but not including the next leader or the end of the program.

Transformation on Basic Blocks

- A number of transformations can be applied to a basic block without changing the set of expressions computed by the block.
- Many of these transformations are useful for improving the quality of the code.
- Types of transformations are:
    1. Structure preserving transformation
    2. Algebraic transformation

Structure Preserving Transformations

- Structure-preserving transformations on basic blocks are:

1. Common sub-expression elimination
2. Dead-code elimination
3. Renaming of temporary variables
4. Interchange of two independent adjacent statements

Common sub-expression elimination

- Consider the basic block,

  a:= b+c

  b:= a-d

  c:= b+c

  d:= a-d

- The second and fourth statements compute the same expression, hence this basic block may be transformed into the equivalent block:

  a:= b+c

  b:= a-d

  c:= b+c

  d:= b

Dead-code elimination

- Suppose x *is* dead, that is, never subsequently used, at the point where the statement $\boldsymbol{x := y + z}$ appears in a basic block.

- Above statement may be safely removed without changing the value of the basic block.

Renaming of temporary variables

- Suppose we have a statement

  t:=b+c, where t is a temporary variable.

- If we change this statement to

  u:= b+c, where u is a new temporary variable,

- Change all uses of this instance of t to u, then the value of the basic block is not changed.

- In fact, we can always transform a basic block into an equivalent block in which each statement that defines a temporary defines a new temporary.

- We call such a basic block a *normal-form* block.

Interchange of two independent adjacent statements

- Suppose we have a block with the two adjacent statements,

  t1:= b+c

  t2:= x+y

- Then we can interchange the two statements without affecting the value of the block if and only if neither x nor y is $t1$ and neither b nor c is $t2$.

- A normal-form basic block permits all statement interchanges that are possible.

Algebraic Transformation

- Countless algebraic transformation can be used to change the set of expressions computed by the basic block into an algebraically equivalent set.

- The useful ones are those that simplify expressions or replace expensive operations by cheaper one.

- Example: x=x+0 or x=x*1 can be eliminated.

A simple code generator

- The code generation strategy generates target code for a sequence of three address statement.
- It uses function getReg() to assign register to variable.
- The code generator algorithm uses descriptors to keep track of register contents and addresses for names.
- **Address descriptor** stores the location where the current value of the name can be found at run time. The information about locations can be stored in the symbol table and is used to access the variables.
- **Register descriptor** is used to keep track of what is currently in each register. The register descriptor shows that initially all the registers are empty. As the generation for the block progresses the registers will hold the values of computation.

Generating a code for assignment statement

- The assignment statement d:= (a-b) + (a-c) + (a-c) can be translated into the following sequence of three address code:

t:= a-b

u:= a-c

v:= t +u

d:= v+u

Statement	Code Generated	Register descriptor	Address descriptor
t:= a - b	MOV a,R0 SUB b, R0	R0 contains t	t in R0
u:= a - c	MOV a,R1 SUB c, R1	R0 contains t R1 contains u	t in R0 u in R1
v:= t + u	ADD R1, R0	R0 contains v R1 contains u	u in R1 v in R0
d:= v + u	ADD R1,R0 MOV R0, d	R0 contains d	d in R0 d in R0 and memory

Code Optimization

- Code Optimization is a program transformation technique which, tries to improve the code by eliminating unnecessary code lines and arranging the statements in such a sequence that speed up the execution without wasting the resources.

1. Faster execution
2. Better performance
3. Improves the efficiency

Code Optimization techniques (Machine independent techniques)

Techniques

1. Compile time evaluation
2. Common sub expressions elimination
3. Code Movement or Code Motion
4. Reduction in Strength
5. Dead code elimination

Compile time evaluation

- Compile time evaluation means shifting of computations from run time to compile time.
- There are two methods used to obtain the compile time evaluation.

Folding

- In the folding technique the computation of constant is done at compile time instead of run time.

  Example : length = (22/7)*d

- Here folding is implied by performing the computation of 22/7 at compile time.

Constant propagation

- In this technique the value of variable is replaced and computation of an expression is done at compilation time.

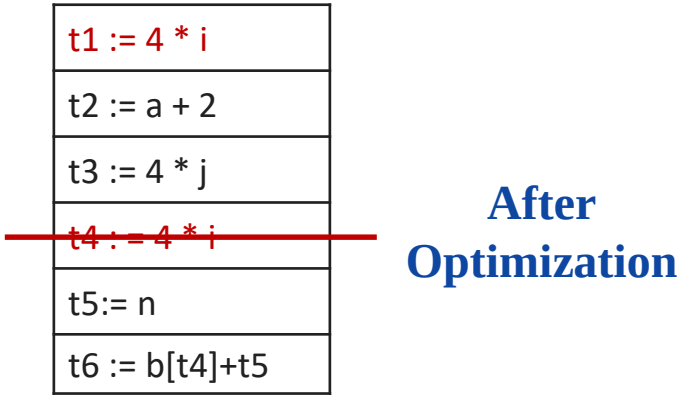
  Example : pi = 3.14; r = 5;

  Area = pi * r * r;

- Here at the compilation time the value of pi is replaced by 3.14 and r by 5 then computation of 3.14 * 5 * 5 is done during compilation.

Common sub expressions elimination

- The common sub expression is an expression appearing repeatedly in the program which is computed previously.
- If the operands of this sub expression do not get changed at all then result of such sub expression is used instead of re-computing it each time.
- Example:



Code Movement or Code Motion

- Optimization can be obtained by moving some amount of code outside the loop and placing it just before entering in the loop.
- It won't have any difference if it executes inside or outside the loop.
- This method is also called loop invariant computation.
- Example:

```
While(i<=max-1)
{
   sum=sum+a[i];
}
```

```
N=max-1;
While(i<=N)
{
    sum=sum+a[i];
}
```

Before Optimization

After Optimization

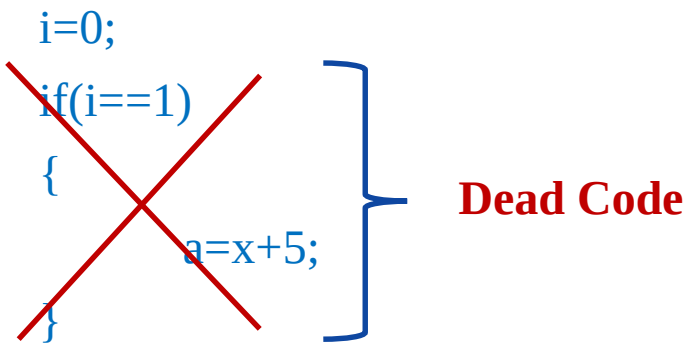
Reduction in Strength

- Priority of certain operators is higher than others.
- For instance strength of * is higher than +.
- In this technique the higher strength operators can be replaced by lower strength operators.
- Example:

A=A*2	A=A+A
Before Optimization	**After Optimization**

Dead code elimination

- The variable is said to be dead at a point in a program if the value contained into it is never been used.
- The code containing such a variable ***is*** supposed to be a dead code.
- Example:



- If statement is a dead code as this condition will never get satisfied hence, statement can be eliminated and optimization can be done.

Machine dependent optimization

- Machine dependent optimization may vary from machine to machine.
- Machine-dependent optimization is done after the **target code** has been generated and when the code is transformed according to the target machine architecture.
- Machine-dependent optimizers put efforts to take maximum **advantage** of the memory hierarchy.

Techniques

1. Register allocation
2. Use of addressing modes
3. Peephole optimization

End of Unit 5

Thank You